

Week 1 Lab 1: Collaborating with Git and GitHub from Google Colab

A hands-on coding lab using Colab, Git Bash and GitHub Desktop, worked in groups

Contents

1. What you will do today	4
How Colab and GitHub fit together	4
By the end of the lab, you can	5
Roles in your group	6
2. Before you start: set-up check	7
2.1 Check Colab	7
2.2 Install the software	7
2.4 Tell Git who you are	7
2.5 Sign in	8
3. Part 1: Create the group repository	9
Person A: create it	9
Everyone else: accept the invitation	9
4. Part 2: clone the repository	10
5. Part 3: from Colab to GitHub through your laptop	11
3a. Open the lab notebook and make it yours	11
3b. Clear outputs and download into your repository	11
3c. Create a branch from an up-to-date main	11
3d. Stage and commit	12
3e. Push the branch to GitHub	12
6. Part 4: pull requests and code review	13
4a. Finish sections 4 and 5, and push again	13

4b. Open your pull request	13
4c. Review the pull request you were given	13
7. Part 5: respond to your review, merge and pull	15
5a. Make the changes in Colab	15
5b. Commit and push to the same branch	15
5c. Merge and bring your copy up to date	15
5d. Open a teammate's notebook from GitHub	15
8. Part 6: the engineered merge conflict	17
6a. Both people make a clashing change	17
6b. The first pull request merges cleanly	17
6c. The resolver brings main into their branch	18
6d. Read the markers and decide	18
6e. Complete the merge and push	18
9. Part 7: save from Colab straight to GitHub	20
7a. Create a branch on GitHub	20
7b. Save from the right tab	20
7c. Save a copy in GitHub	20
7d. Merge it and bring it to your laptop	20
The two routes compared	20
10. Part 8: set up your personal ST445 repository	22
8a. Create and clone it	22
8b. Create the folder structure	22
8c. Extend the .gitignore	22
8d. Record the environment from Colab	23
8e. Write the README	23
8f. Add the data dictionary	24
8g. Commit, push and ask for review	24
11. Formative: the repository health check	25
12. Extension tasks	26
Draw the history	26
Clone the repository inside Colab	26
Strip outputs automatically	26
Protect main	26
Undo safely	26
Link issues to pull requests	26
13. When something goes wrong	27
14. Quick reference	29
Colab	29

Git.....	29
Before you leave	30
Appendix: notes for tutors.....	Error! Bookmark not defined.

1. What you will do today

Your group will use GitHub to build and change a shared repository together. For this, you will write Python code in Google Colab. By the end, you will have moved a Colab notebook into a repository, committed and pushed it on a branch, opened and reviewed a pull request, acted on the review, and resolved a merge conflict. After the seminar, you save a notebook from Colab straight to GitHub and set up your personal ST445 repository for the rest of the module.

For this lab, you will need this tutorial and the Jupyter notebook as described in Table 1. This tutorial explains each Git and GitHub step for both Git Bash and GitHub Desktop. The lab notebook, [st445_week1_lab1.ipynb](#), is the file you work on: it has Python exercises and five **Git checkpoints** that tell you when to stop and do the next part of this tutorial. Open the notebook from the course GitHub repository (the link is on Moodle) using its **Open in Colab** badge.

When	Tutorial	Lab notebook	Branch
Seminar	Parts 1 and 2: create and clone the group repository	none yet	<code>main</code>
Seminar	Part 3: commit and push	Sections 1 to 3, checkpoint A	<code>lab-<yourname></code>
Seminar	Part 4: pull request and review	Sections 4 and 5, checkpoint B	<code>lab-<yourname></code>
Seminar	Part 5: respond, merge and pull	Checkpoint C	<code>lab-<yourname></code>
Seminar	Part 6: the engineered merge conflict	Checkpoint D	<code>title-<yourname></code> <code>or day-<yourname></code>
After	Part 7: save straight from Colab to GitHub	Sections 6 to 10, checkpoint E	<code>colab-<yourname></code>
After	Part 8 and the formative: your personal repository	Files from section 9, checkpoint E	<code>setup</code>

Table 1. Week 1 Seminar Lab Steps

Every step has its own branch, and each branch is deleted once it is merged, so a name is never reused.

How Colab and GitHub fit together

Colab runs your notebooks in the cloud and saves them to your Google Drive, in a folder called Colab Notebooks. Git and GitHub know nothing about that copy. Your repository lives in a folder on your laptop, and Git only tracks the files in that folder. So, there are two routes to move your Colab notebooks to GitHub:

Route	How it works	Use it when
Through your laptop (main route, Parts 3 to 6 and 8)	In Colab, File → Download → Download .ipynb . Put the file in your local repository folder, then stage, commit and push with Git Bash or GitHub Desktop.	Almost always: you can commit several files together, check the changes before committing, use <code>.gitignore</code> , and resolve conflicts.
Straight from Colab (Part 7)	In Colab, File → Save a copy in GitHub . Colab makes one commit on a branch you choose.	A quick save of a single notebook when you do not need to review the change or commit anything else with it.

Table 2. Routes to use Git for this tutorial.

One rule for the whole module

The copy in your GitHub repository is the one that counts. Whenever you change a notebook in Colab, download it again and replace the file in your repository folder before you commit. A change that exists only in Google Drive is invisible to your teammates and to your markers.

Every Git task is written for both tools. **Git Bash** (on macOS, the Terminal app) is the command line: it is what most documentation, Stack Overflow answers, and chatbots assume. **GitHub Desktop** is a graphical app that does the same things with buttons. They work on exactly the same repository, so group members can use different tools and still collaborate. You will use both before the end of the session.

By the end of the lab, you can

- clone a repository from GitHub to your laptop
- download a notebook from Colab into your repository, then stage, commit and push it
- create a branch, open a pull request, review a teammate's code and merge it
- act on a review and bring the new version of a notebook back into your repository
- bring your copy up to date with `git pull`, and open a teammate's notebook from GitHub in Colab
- resolve a merge conflict and complete the merge
- and, after the seminar, save a notebook from Colab straight to a branch on GitHub, and set up a repository with a README, a `.gitignore`, a folder structure, a requirements file and a data dictionary

Roles in your group

Each person in your group should receive a letter, as shown in Table 2. The letters decide who does what in Parts 1, 4, and 6; everyone should do everything else.

Person	Role	Reviews in Part 4	Tool for Parts 2 to 5	Tool for Part 6
A	Maintainer: creates the repository and adds the others	B	Git Bash	GitHub Desktop
B	Collaborator	C	GitHub Desktop	Git Bash
C	Collaborator	D (or A in a group of three)	Git Bash	GitHub Desktop
D	Collaborator (groups of four)	A	GitHub Desktop	Git Bash

Table 3. Group roles and revision pairing

Everyone writes Python in Colab; the tool in the table is the one you use for Git. Give the maintainer role to someone who has used Git before if your group has one. In a group of three, Person A also does Person D's part in the Part 6 conflict.

2. Before you start: set-up check

Ensure that you have a Colab account and either Git or GitHub Desktop installed on your personal computer. You must also set them up and make sure they're ready to use.

2.1 Check Colab

Go to colab.research.google.com, sign in with your Google account and create a new notebook (**File** → **New notebook**). Run a cell containing `print("hello")`. Close the notebook; it has been saved in the **Colab Notebooks** folder of your Google Drive. You open the lab notebook itself in Part 3.

2.2 Install the software

What	Windows	macOS
Git	Install Git for Windows from git-scm.com . Accept the defaults, except choose VS Code as the default editor when asked. This also installs Git Bash.	Open Terminal and run <code>git --version</code> . If Git is missing, macOS offers to install the command line developer tools; accept.
GitHub Desktop	desktop.github.com	desktop.github.com

Table 4. Installing Git Bash and Git Desktop

On Linux, use your terminal for every Git Bash step. GitHub Desktop has no official Linux version, so take the Git Bash column in Part 6 as well. If your laptop cannot be set up today, open a GitHub Codespace from any repository (the green **Code** button, then the **Codespaces** tab); its terminal has Git ready to use, and you can upload a downloaded notebook into it by dragging it into the Explorer.

2.4 Tell Git who you are

To use Git, you must set up your account first. Run these once in Git Bash or Terminal, using the email address on your GitHub account:

```
git config --global user.name "Your Name"
git config --global user.email "you@lse.ac.uk"
git config --global init.defaultBranch main
```



The screenshot shows a Windows terminal window titled "MINGW64:/c/Users/ALVESDIA". The prompt is "LSE+ALVESDIA@LSE-BHRWCN4 MINGW64 ~". The user enters the following commands:

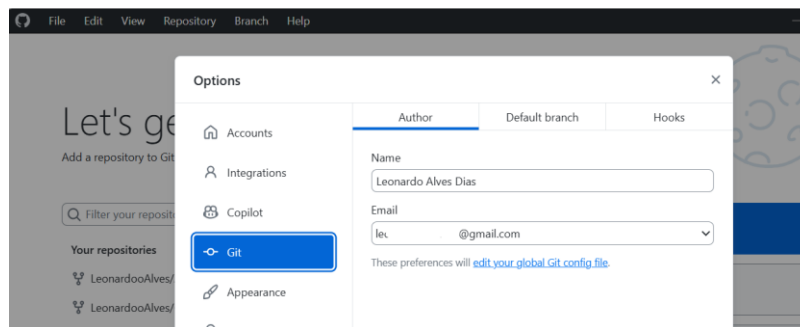
```
$ git config --global user.name "LeonardooAlves"
$ git config --global user.email "leo@gmail.com"
$ git config --global init.defaultBranch main
```

If you have VS Code on your machine, you can also run the commands below to use VS Code rather than the Vim editor when Git needs you to write a message.

```
git config --global core.editor "code --wait"
git config --global --list      # check what you set
```

2.5 Sign in

- **GitHub Desktop:** open it and choose **Sign in to GitHub.com**. A browser window opens; approve it, including your two-factor code. On Windows, the settings are under **File → Options**; on macOS, under **GitHub Desktop → Settings**.



- **Git Bash on Windows:** nothing to do yet. The first time you push, Git Credential Manager (installed with Git for Windows) opens a browser window to sign you in, and remembers you afterwards.
- **Terminal on macOS:** GitHub no longer accepts your account password on the command line. The simplest route is the GitHub CLI: install it (`brew install gh`, or the installer from cli.github.com), run `gh auth login`, choose GitHub.com and HTTPS, and answer yes when asked to authenticate Git with your GitHub credentials.
- **Colab:** nothing to do yet. The first time you open or save a notebook on GitHub from Colab (Parts 4 and 7), it asks you to authorise Colab to use your GitHub account.

Checkpoint

A Colab notebook runs `print("hello")`.

`git --version` prints a version number.

`git config --global user.email` prints the email on your GitHub account.

GitHub Desktop shows your GitHub username under its account settings.

3. Part 1: Create the group repository

Who: Person A. Everyone else watches the screen and gets ready to accept an invitation.

Person A: create it

1. On github.com, click + at the top right, then **New repository**.
2. Name it `st445-week1-group<N>`, using your group number.
3. Set it to **Private**.
4. Tick **Add a README file**. Under **Add .gitignore**, choose **Python**.
5. Click **Create repository**.
6. Click **Add file** → **Create new file**, name it `team/meeting_notes.md`, and type exactly these two lines:

```
# Group meeting notes
Meeting day: to be agreed
```

7. Click **Commit changes**, keep **Commit directly to the main branch**, and click **Commit changes** again.
8. Go to **Settings** → **Collaborators** and click **Add people**. Add each teammate by GitHub username.

Everyone else: accept the invitation

Open the invitation email from GitHub, or go to github.com/notifications, and click **Accept invitation**. Until you accept, you can see nothing, and your pushes will be refused.

Checkpoint

Every member can open the repository page on GitHub.

The repository contains `README.md`, `.gitignore` and `team/meeting_notes.md`.

4. Part 2: clone the repository

Who: everyone, each on their own laptop, with the tool in the roles table.

Cloning copies the whole repository, including its history, from GitHub to a folder on your laptop. This folder is where every file you want in the repository must end up, including notebooks downloaded from Colab. Copy the repository address first: on GitHub, click the green **Code** button and copy the HTTPS URL.

Git Bash / Terminal

```
# go to where you keep coursework
cd ~/Documents

git clone https://github.com/<owner>/st445-week1-group<N>.git
cd st445-week1-group<N>
mkdir notebooks

git status
git log --oneline
```

GitHub Desktop

1. **File → Clone repository.**
2. On the **GitHub.com** tab, choose **st445-week1-group<N>**.
3. Check the **Local path**, for example **Documents**.
4. Click **Clone**.
5. Choose **Repository → Show in Explorer** (Windows) or **Show in Finder** (macOS) and create a folder called **notebooks** there.
6. Click **History** to see the commits so far.

`git status` should report that you are on branch **main** and that the working tree is clean. The empty **notebooks** folder does not count as a change: Git only tracks files, so it notices the folder once a notebook is in it.

Note the full path of your repository folder, for example **Documents/st445-week1-group3**. You will save downloads there in the next part.

Where is the change?

Keep the three places from the lecture in mind. The **working directory** is the files in your repository folder. The **staging area** holds the changes you have chosen for the next commit. The **repository** is the history of commits. Pushing copies commits from your laptop to GitHub. A notebook in Google Drive is in none of these places until you download it into the folder.

Checkpoint

Each member has a folder **st445-week1-group<N>** on their laptop, with an empty **notebooks** folder inside.

History shows two commits: the initial commit and the meeting notes.

5. Part 3: from Colab to GitHub through your laptop

Who: everyone, at the same time. **Notebook:** sections 1 to 3, then checkpoint A.

You never work directly on `main` in this module. You make a branch, commit to it, and ask for it to be merged through a pull request. Each of you works on your own copy of the lab notebook. Because every copy has a different file name, nobody collides in this part.

3a. Open the lab notebook and make it yours

1. Open the course notebooks repository (link on Moodle) and click the **Open in Colab** badge next to `st445_week1_lab1.ipynb`. Colab opens it straight from GitHub, read-only.
2. Choose **File** → **Save a copy in Drive**. Work only in this copy from now on.
3. Click the title at the top left and rename your copy `<yourname>_week1_lab.ipynb`, for example `priya_week1_lab.ipynb`. The downloaded file takes this name.
4. Run the two setup cells, then work through sections 1 to 3. Stop when you reach **Git checkpoint A**.

3b. Clear outputs and download into your repository

1. Choose **Edit** → **Clear all outputs**. Outputs change every time a notebook runs, which fills the repository history with noise, and they can contain data you did not mean to publish.
2. Choose **File** → **Download** → **Download .ipynb**.
3. In the save dialog, go to your repository folder, open `notebooks`, and save. (If the file went to Downloads instead, move it into `notebooks` now.)
4. Check the name is exactly `priya_week1_lab.ipynb`. If your browser added `(1)`, rename it.

The notebook is now in your working directory. Nothing has been staged or committed yet.

3c. Create a branch from an up-to-date main

Name it `lab-<yourname>`, for example `lab-priya`. You use this one branch for Parts 3, 4 and 5.

Git Bash / Terminal

```
git switch main
git pull
git switch -c lab-priya
git branch          # * marks your branch
```

GitHub Desktop

1. Click **Current branch** and choose `main`.
2. Click **Fetch origin**, then **Pull origin** if it appears.
3. Click **Current branch** → **New branch**, type `lab-priya`, click **Create branch**.
4. When asked, bring your changes to the new branch.

3d. Stage and commit

Git Bash / Terminal

```
git status    # untracked: notebooks/  
git add notebooks/priya_week1_lab.ipynb  
git status    # to be committed  
git commit -m "Complete lab sections 1 to 3"  
git log --oneline
```

GitHub Desktop

1. The **Changes** tab lists `notebooks/priya_week1_lab.ipynb` with a tick. The tick means it will be staged in the commit.
2. Click the file to see what is added. A notebook is a JSON text file, so you see its structure rather than cells.
3. Bottom left, type a **Summary**: `Complete lab sections 1 to 3`.
4. Click **Commit to lab-priya**.

A good commit message is a short summary in the imperative ("Add", "Complete", "Fix") saying what changed and, if it is not obvious, why. `update` and `stuff` tell a reader nothing. Section 6 of the notebook has you write a checker for exactly this.

3e. Push the branch to GitHub

Git Bash / Terminal

```
git push -u origin lab-priya  
  
# -u links your branch to the one on  
# GitHub, so later you can type just  
# git push
```

GitHub Desktop

1. Click **Publish branch** at the top.
2. Later pushes on this branch use the same button, now labelled **Push origin**.

On GitHub, switch to your branch and click your notebook: GitHub displays it as a notebook, with cells. Then go back to your Colab tab and carry on with section 4. The tab still holds exactly the version you pushed.

Checkpoint

On GitHub, the branch menu shows one `lab-<name>` branch per member.

Each branch contains that member's notebook in `notebooks/`, with no outputs; `main` does not yet.

Navigator question: after the download but before the commit, could your teammates see your notebook? (No: it was only in your working directory. Notebook exercise 7 models exactly this.)

6. Part 4: pull requests and code review

Who: everyone. Review pairs are in the roles table: A reviews B, B reviews C, C reviews D (or A in a group of three), D reviews A. **Notebook:** sections 4 and 5, then checkpoint B.

A pull request is a proposal: "please merge my branch into `main`". It shows exactly what would change and lets teammates comment on specific lines before anything is merged.

4a. Finish sections 4 and 5, and push again

Work through sections 4 and 5 of the notebook. They include the two functions your reviewer will read: `file_stage` and `conflict_versions`. Then clear outputs, download the notebook into `notebooks/` replacing the old file (same name), and commit and push to the same branch.

Git Bash / Terminal

```
git status      # modified: notebooks/...
git diff --stat
git add notebooks/priya_week1_lab.ipynb
git commit -m "Complete lab sections 4 and 5"
git push
```

GitHub Desktop

1. The **Changes** tab shows the notebook as modified.
2. Write the summary `Complete lab sections 4 and 5` and click **Commit to lab-priya**.
3. Click **Push origin**.

4b. Open your pull request

Git Bash / Terminal

```
# the first push printed a link;
# open it, or on GitHub click
# "Compare & pull request"

# optional, with the GitHub CLI:
gh pr create --base main \
  --title "Add Priya's week 1 lab" \
  --body "Sections 1 to 5."
```

GitHub Desktop

1. Click **Preview Pull Request** or **Create Pull Request** (or **Branch → Create pull request**).
2. GitHub opens in your browser with the pull request form.

On the form, check that **base** is `main` and **compare** is `lab-<yourname>`. Give it a title and one sentence of description. Under **Reviewers** on the right, choose your reviewer from the roles table. Click **Create pull request**.

4c. Review the pull request you were given

1. Open the **Pull requests** tab and click the one you are reviewing.
2. Click **Files changed**. GitHub may show the notebook rendered as cells; if you cannot comment on a line, switch to the source view of the diff.
3. Leave at least two line comments. Hover over a line and click the blue **+** that appears, then **Start a review**:
 - on `file_stage`: is every code handled, including the space in " `M`" and "`M` "? Could it be shorter with a dictionary?
 - on `conflict_versions`: does it handle a conflict with several lines on each side?

4. Click **Review changes** (top right), choose **Request changes** (or **Approve** if there is genuinely nothing to change), and submit.

A useful comment is specific, about the code rather than the person, and suggests what to do: "This returns `unknown` for 'A ', a new staged file; could you add that case?"

Do not merge yet: you respond to your review in Part 5.

Checkpoint

Every member has an open pull request from `lab-<name>` with a reviewer assigned.

Every member has left at least two line comments on the pull request they were given.

7. Part 5: respond to your review, merge and pull

Who: everyone. **Notebook:** checkpoint C.

5a. Make the changes in Colab

Your Colab tab still holds the version you pushed in Part 4, so make the changes your reviewer asked for there. If you closed the tab, use the **round trip** instead: **File** → **Upload notebook**, **Browse**, and choose `notebooks/priya_week1_lab.ipynb` from your repository folder. Never reopen an older copy from Drive: it may not be the version your reviewer saw.

Run the test cells to check your fix, then **Edit** → **Clear all outputs** and download the notebook into `notebooks/`, replacing the old file.

5b. Commit and push to the same branch

Git Bash / Terminal

```
git diff --stat # your fix only
git add notebooks/priya_week1_lab.ipynb
git commit -m "Handle multi-line conflicts in
conflict_versions"
git push
```

GitHub Desktop

1. Check the **Changes** tab shows the notebook as modified.
2. Write a summary that says what you changed and click **Commit to lab-priya**.
3. Click **Push origin**.

The pull request updates itself; you do not open a new one. Reply to each comment ("Done in the latest commit"), click **Resolve conversation**, and ask your reviewer to click **Review changes** → **Approve**.

5c. Merge and bring your copy up to date

Once approved, click **Merge pull request**, **Confirm merge**, then **Delete branch**. Deleting it on GitHub is safe: its commits are now part of `main`. Then bring your laptop up to date. `git pull` fetches new commits from GitHub and merges them into your current branch.

Git Bash / Terminal

```
git switch main
git pull
ls notebooks # everyone's notebooks

# tidy up your merged branch
git branch -d lab-priya
```

GitHub Desktop

1. Click **Current branch** and choose `main`.
2. Click **Fetch origin**; it changes to **Pull origin**. Click it.
3. Right-click `lab-priya` in the branch list and choose **Delete**.

5d. Open a teammate's notebook from GitHub

Colab can open any notebook in a repository you can see. Choose **File** → **Open notebook**, click the **GitHub** tab, and tick **Include private repos** (the first time, Colab asks you to authorise it on GitHub; approve it). Choose the group repository and `main`, open a teammate's notebook, compare their `file_stage` with yours, and close it without saving.

Habit to keep

Always switch to `main` and pull before starting a new branch. Most conflicts in group work come from branching off an old copy of `main`.

Checkpoint

Every pull request is reviewed, approved and merged, and every `lab-<name>` branch is deleted.

On GitHub, `main` contains one lab notebook per member in `notebooks/`, with no outputs.

Everyone's laptop is on an up-to-date `main`.

8. Part 6: the engineered merge conflict

Who: two pairs. Pair 1 is B and C; pair 2 is A and D. **Swap tools now:** use the one in the Part 6 column of the roles table. **Notebook:** checkpoint D.

A merge conflict happens when two branches change the same lines of the same file. Git will not guess which version is right, so it stops and asks a person to decide. This is Git working correctly. You are going to cause one on purpose. You have already met the markers: `conflict_versions` in notebook section 5 reads exactly the lines you are about to resolve.

Why this part uses text files, not notebooks

A notebook is one long JSON file. When two people edit the same notebook, the conflict markers land inside the JSON, and the file will not open in Colab until the JSON is repaired by hand. In group work, avoid the problem: give each person their own notebook, as you did in Part 3, and share code through separate files. Conflicts in text files such as `README.md` are easy to read and fix, so that is what you practise here, editing the files in VS Code rather than Colab.

Pair	File	Line to change
1: B and C	<code>README.md</code>	Line 1, the title. Each of you writes a different project title, for example <code># London A&E waiting times</code> and <code># Housing affordability in England</code> .
2: A and D	<code>team/meeting_notes.md</code>	Line 2. Each of you writes a different day, for example <code>Meeting day: Tuesday</code> and <code>Meeting day: Thursday</code> .

6a. Both people make a clashing change

Each of you, separately: make sure you are on an up-to-date `main` (Part 5), create a branch called `title-<yourname>` or `day-<yourname>`, open the file in Notepad or VS Code, change your pair's line to your own version and save, then commit, push and open a pull request, exactly as in Parts 3 and 4.

6b. The first pull request merges cleanly

In each pair, whoever opened their pull request first merges it (skip the review this time). The second person's pull request now shows **This branch has conflicts that must be resolved**. That person is the **resolver**; the other becomes their navigator.

6c. The resolver brings main into their branch

Git Bash / Terminal

```
git switch main
git pull
git switch title-sam
git merge main

# CONFLICT (content): Merge conflict
# in README.md
git status      # both modified: README.md
```

GitHub Desktop

1. Make sure `title-sam` is the **Current branch**.
2. Click **Fetch origin**.
3. Choose **Branch** → **Update from main**.
4. A dialogue says there are conflicts and lists `README.md`.
5. Click **Open in Visual Studio Code** next to the file.

6d. Read the markers and decide

Open the file in Notepad or VS Code. Git has written both versions into it:

```
<<<<<< HEAD
# Housing affordability in England
=====
# London A&E waiting times
>>>>>> main
```

- Between `<<<<<< HEAD` and `=====` is the version on your current branch.
- Between `=====` and `>>>>>> main` is the version coming in from `main`.

Agree with your partner on which version to keep or write a combination. Notepad/VS Code shows buttons above the block: **Accept Current Change**, **Accept Incoming Change** and **Accept Both Changes**. Whatever you choose, all three marker lines must be gone, and the file must read correctly. Save it.

6e. Complete the merge and push

Git Bash / Terminal

```
git add README.md
git commit      # VS Code opens with a
                # message; save, close
git push
```

GitHub Desktop

1. Back in GitHub Desktop, the file now shows as resolved and **No conflicts remaining** appears.
2. Click **Continue merge**.
3. Click **Push origin**.

Refresh the pull request on GitHub. The conflict warning is gone, and it can be merged. Merge it and delete the branch.

Resolving on the website

For a small conflict in a text file, GitHub also offers a **Resolve conflicts** button on the pull request, which opens the same markers in a web editor. It is convenient, but learn the laptop route first: it is the one that works for every kind of conflict.

Checkpoint

Both pull requests in each pair are merged.

`README.md` and `team/meeting_notes.md` on GitHub contain no `<<<<<<`, `=====` or `>>>>>>` lines.

In GitHub Desktop **History**, or with `git log --oneline --graph`, you can see the merge commit joining two lines of history.

9. Part 7: save from Colab straight to GitHub

Who: everyone, individually. **When:** after the seminar. **Notebook:** sections 6 to 10, then checkpoint E.

Finish sections 6 to 10 of the lab notebook in your Colab copy. Colab can then commit the notebook to GitHub itself, without your laptop.

7a. Create a branch on GitHub

Colab can only save to a branch that already exists on GitHub, so create it there. On the group repository page, click the branch menu (it says `main`), type `colab-<yourname>`, and click **Create branch: colab-<yourname> from main**.

7b. Save from the right tab

Save from the Colab tab that holds your sections 6 to 10 work. If you closed it, reopen your Drive copy with **File → Open notebook → Google Drive**, not the version on GitHub, which stops at section 5. Then choose **Edit → Clear all outputs**: saving to GitHub from Colab commits whatever is in the notebook, outputs included.

7c. Save a copy in GitHub

1. Choose **File → Save a copy in GitHub**.
2. In the dialog, check the **Repository** is your group repository and set **Branch** to `colab-<yourname>`. Never choose `main`: that would skip the pull request.
3. Set the **File path** to `notebooks/<yourname>_week1_lab.ipynb`, so the save replaces your existing file rather than creating a second one.
4. Write a **Commit message** in the usual style: `Complete lab sections 6 to 10`.
5. Untick **Include a link to Colaboratory**. If left ticked, Colab adds an "Open in Colab" badge cell to the notebook, which is not part of your work.
6. Click **OK**. Colab makes the commit and opens the file on GitHub.

7d. Merge it and bring it to your laptop

On GitHub, open a pull request from `colab-<yourname>` into `main`, merge it (a review is optional here), and **delete the branch**. Then on your laptop, switch to `main` and pull, as in Part 5. The work you did in Colab is now in your repository folder too.

The two routes compared

Question	Through your laptop	Straight from Colab
Commits	As many files per commit as you choose	One notebook per commit
Checking the change first	You see the diff before you commit	No diff: it commits immediately

Question	Through your laptop	Straight from Colab
Other files	<code>.gitignore</code> , README, data folders, anything	Only the notebook you are saving
Conflicts	Resolved on your laptop	Not handled; the save simply replaces the file on that branch
Good for	Everyday work	A quick save of one notebook to its own branch

Table 5. Differences of using Git in your laptop vs Git in Google Colab

Checkpoint

Your notebook on `main` on GitHub has all ten sections and no outputs.

There is no "Open in Colab" badge cell at the top.

The `colab-<yourname>` branch is deleted, and after `git pull` the same version is in your repository folder.

10. Part 8: set up your personal ST445 repository

Who: everyone, individually, on your own repository. **When:** after the seminar. **Notebook:** the files you made in section 9, and checkpoint E.

This is the repository you will use for the whole module, and it is what your partner reviews in this week's formative. If your tutor gives you a link to the module template repository, click **Use this template** → **Create a new repository**, clone it, and skip to step 8c. Otherwise, build it as follows.

8a. Create and clone it

On GitHub, create a **Private** repository named `st445-<yourusername>`, with a README and the **Python .gitignore**. Clone it with either tool, as in Part 2, then create a branch called `setup` for everything that follows.

8b. Create the folder structure

Git does not track empty folders, so each one gets an empty placeholder file called `.gitkeep`.

Git Bash / Terminal

```
mkdir -p data/raw data/processed \  
notebooks src outputs/figures  
  
touch data/raw/.gitkeep \  
data/processed/.gitkeep \  
notebooks/.gitkeep src/.gitkeep \  
outputs/figures/.gitkeep
```

GitHub Desktop

1. Choose **Repository** → **Open in Visual Studio Code**.
2. In the VS Code Explorer, create the folders `data/raw`, `data/processed`, `notebooks`, `src` and `outputs/figures`.
3. In each folder, create an empty file named `.gitkeep`.

The result should look like this:

```
st445-<yourusername>/  
├── README.md  
├── .gitignore  
├── requirements.txt      package versions used in Colab  
├── data/  
│   ├── data_dictionary.md  what each column means  
│   ├── raw/                original data, never edited  
│   └── processed/          rebuilt by your code  
├── notebooks/             notebooks downloaded from Colab  
├── src/  
├── outputs/  
└── figures/
```

8c. Extend the .gitignore

These are the rules you wrote in notebook section 9. Open `.gitignore` in VS Code and paste them at the **top**. Do not replace the file with the one from Colab: it already holds GitHub's Python rules, such as `__pycache__` and `.ipynb_checkpoints`, which you want to keep. The `!` lines keep the placeholders while ignoring the data inside the folders.

```
# secrets: never commit API keys
.env

# data stays out of the repository
data/raw/*
data/processed/*
!data/raw/.gitkeep
!data/processed/.gitkeep

# system files
.DS_Store
```

An API key that reaches GitHub should be treated as compromised even if you delete it afterwards, because it stays in the history. This matters from Week 2, when you start calling APIs from Colab: keep keys in Colab's **Secrets** panel (the key icon in the left sidebar), never in a notebook cell.

8d. Record the environment from Colab

Colab's package versions change over time, so a notebook that runs today may behave differently next term. Record the versions you used. The requirements cell in section 9 of the lab notebook does this and downloads the file; if you no longer have it, run the same code in any Colab notebook:

```
import sys, numpy, pandas, pyarrow
print("Python", sys.version.split()[0])

with open("requirements.txt", "w") as f:
    for m in (numpy, pandas, pyarrow):
        f.write(f"{m.__name__}=={m.__version__}\n")

from google.colab import files
files.download("requirements.txt")
```

The last two lines download the file through your browser; save it at the top of your repository folder. (You can also download any file from Colab's **Files** panel, the folder icon in the left sidebar: right-click the file and choose **Download**.) Add packages to the list as the module introduces them, and note the Python version in your README.

8e. Write the README

Replace the contents of `README.md` with this outline and fill it in. It will grow over the term.

```
# ST445 Managing and Visualising Data: <your name>

Coursework repository for ST445, Autumn Term 2026/27.

## How to run
1. Open a notebook from `notebooks/` in Google Colab:
   File > Open notebook > GitHub, then choose this repository.
2. Package versions are in `requirements.txt` (Python 3.x).
   If Colab's versions differ, install them in the first cell:
   `!pip install pandas==<version>`
3. Run the notebooks in numbered order.

## Structure
- `data/raw/`: original data, never edited (not in the repository)
- `data/processed/`: produced by the code
- `notebooks/`, `src/`: analysis code
- `outputs/figures/`: figures produced by the code

## Data sources
To be added as data is collected (Week 2).
```

8f. Add the data dictionary

A data dictionary says what each column in your data means, its type and units, and how missing values are recorded. Save the `data_dictionary.md` you downloaded in notebook section 9 as `data/data_dictionary.md`. It is tracked even though the data folders are ignored, because the ignore rules cover only what is inside `data/raw/` and `data/processed/`. For now it describes the example extract; from Week 2 you replace it with your own data's columns.

8g. Commit, push and ask for review

1. Commit your work on the `setup` branch in small, well-described commits, for example "Add project folder structure", "Ignore data and secrets", "Add requirements from Colab", "Add data dictionary", "Write README".
2. Push the branch and open a pull request into `main`.
3. In **Settings** → **Collaborators**, add your review partner, then request them as the reviewer on the pull request.

11. Formative: the repository health check

Your partner reviews your `setup` pull request, and you review theirs, against the four criteria below. Leave at least one comment per criterion in the review, on the relevant line where you can, and finish with **Approve** or **Request changes**. The author then responds, pushes any fixes and merges.

This does not count towards your mark. The reproducibility criterion is the same one marked in the coursework project, so this is your first rehearsal of it. Your tutor samples five repositories and posts an annotated example of a good one.

Criterion	Met when	Not yet met when
Structure	The folders match Part 8 and a stranger could guess where anything belongs.	Files sit at the top level, or data is mixed in with code.
README quality	The README says what the repository is, how to open and run the notebooks in Colab, and where the data comes from (or that this will be added); <code>data/data_dictionary.md</code> describes each column.	It is the default one-line README, the steps would not work, or there is no data dictionary.
Commit messages	Each says what changed in the imperative; each commit is one logical change.	Messages such as <code>update</code> , <code>fix</code> , or one commit containing everything.
Reproducibility	<code>requirements.txt</code> lists pinned versions, data and secrets are ignored, and notebooks are committed without outputs.	No requirements file, a <code>.env</code> or data file committed, or notebooks committed with outputs.

Table 6. List of elements you should consider to enhance the reproducibility of your GitHub repository.

Score one point for each criterion met, out of four.

12. Extension tasks

For anyone who finishes early. Each one is useful later in the module.

Draw the history

Run `git log --oneline --graph --all` in the group repository and find the merge commits from Part 6 and the commit Colab made in Part 7.

Clone the repository inside Colab

A Colab session can run Git itself. In a code cell, `!git clone https://github.com/<owner>/st445-week1-group<N>.git` works for a public repository. Private repositories need a personal access token, which must be kept in Colab's **Secrets** panel, never typed into a cell. Try it on a public test repository, and note that the clone disappears when the Colab session ends.

Strip outputs automatically

If you also have Python on your laptop, `pip install nbstripout` then `nbstripout --install` inside the repository makes Git remove notebook outputs on every commit, so you no longer need to clear them by hand in Colab.

Protect main

In the group repository, Person A opens **Settings → Branches** (or **Rules → Rulesets**), adds a rule for `main` requiring a pull request with one approval before merging, and the group checks that a direct push to `main` (including a Colab save to `main`) is now refused. Branch protection on a private repository may need a paid plan; if the option is unavailable, try it on a public test repository.

Undo safely

Make a commit you regret on a branch, then undo it with `git revert <hash>`. Revert adds a new commit that reverses the old one, which is safe once work has been pushed. Compare it with `git restore <file>`, which discards changes you have not yet committed.

Link issues to pull requests

Open an issue in the group repository ("Agree a meeting day"), assign it to someone, and close it from a pull request by writing `Closes #<issue number>` in the pull request description. When the pull request merges, the issue closes itself.

13. When something goes wrong

You see	What it means	What to do
The notebook is not in <code>git status</code>	It went to your Downloads folder, or to the wrong folder.	Move it into <code>notebooks/</code> in your repository folder. Set the browser to ask where to save (section 2.3).
The downloaded file is <code>Untitled0.ipynb</code> or <code>priya_week1_lab (1).ipynb</code>	The notebook was not renamed in Colab, or the browser avoided overwriting an existing file.	Rename the file so it matches the one in the repository, replacing the old one.
Your change is in Colab but not in the pull request	You edited the Drive copy and never downloaded it into the repository.	Download again into <code>notebooks/</code> , replacing the file, then commit and push.
Colab cannot see your repository in Open notebook → GitHub	It is private and Colab has not been authorised for private repositories.	Tick Include private repos and approve the authorisation on GitHub.
Your branch is not listed in Save a copy in GitHub	The branch exists only on your laptop, or was created after the dialog opened.	Push it, or create it on GitHub (Part 7a), then reopen the dialog.
A notebook will not open after a merge	Conflict markers are inside the notebook's JSON.	Ask your tutor; in future, one notebook per person (Part 6).
<code>fatal: a branch named ... already exists</code>	You reused a branch name, or did not delete a merged branch.	Delete the old branch (<code>git branch -d <name></code> , or right-click it in GitHub Desktop), or check the branch table in section 1.
A notebook test cell gives <code>NameError</code>	The cell that defines the function has not been run in this session.	Run the cells above it, or Runtime → Run before .
Your Part 7 save lost sections 6 to 10	You reopened the notebook from GitHub, which stops at section 5, instead of your Drive copy.	Reopen your Drive copy and save again to the same branch.
<code>fatal: not a git repository</code>	You are in the wrong folder.	<code>cd</code> into the repository folder. <code>pwd</code> shows where you are.
<code>Author identity unknown</code>	Git does not know your name and email.	Run the two <code>git config</code> lines in section 2.4.

You see	What it means	What to do
Permission denied or 403 on push	You have not accepted the invitation, or you are signed in as someone else.	Accept the invitation (Part 1). In GitHub Desktop, check the account in settings.
Support for password authentication was removed	You typed your GitHub password on the command line.	Sign in through the browser (Windows) or <code>gh auth login</code> (macOS), section 2.5.
Updates were rejected because the remote contains work that you do not have locally	Someone pushed to this branch after you last pulled, possibly a Colab save.	<code>git pull</code> , resolve any conflict, then <code>git push</code> again.
A strange full-screen editor opens after <code>git commit</code>	Git opened Vim.	Press Esc, type <code>:wq</code> and press Enter. Then set VS Code as the editor, section 2.4.
warning: LF will be replaced by CRLF	Windows and macOS end lines differently; Git is converting.	Harmless. Carry on.
A file you ignored still appears in <code>git status</code>	It was committed before you added it to <code>.gitignore</code> .	<code>git rm --cached <file></code> , then commit.

Table 7. Troubleshoot guide.

14. Quick reference

Colab

To do this	In Colab
Keep an editable copy of a course notebook	File → Save a copy in Drive
Download a notebook	File → Download → Download .ipynb
Download any other file	Files panel → right-click → Download, or <code>files.download("name")</code>
Open a notebook from your laptop	File → Upload notebook
Open a notebook from GitHub	File → Open notebook → GitHub tab
Commit a notebook straight to GitHub	File → Save a copy in GitHub
Remove outputs before saving	Edit → Clear all outputs

Table 8. Colab Actions.

Git

To do this	Git Bash / Terminal	GitHub Desktop
Copy a repository to your laptop	<code>git clone <url></code>	File → Clone repository
See where your changes are	<code>git status</code>	Changes tab
Create a branch and move onto it	<code>git switch -c <name></code>	Current branch → New branch
Move to another branch	<code>git switch <name></code>	Current branch → choose it
Stage a change	<code>git add <file></code>	Tick the file in Changes
Commit	<code>git commit -m "<message>"</code>	Summary, then Commit to <branch>
Send commits to GitHub	<code>git push</code> (first time: <code>git push -u origin <branch></code>)	Publish branch / Push origin
Get new commits from GitHub	<code>git pull</code>	Fetch origin, then Pull origin

To do this	Git Bash / Terminal	GitHub Desktop
Bring main into your branch	<code>git merge main</code>	Branch → Update from main
See the history	<code>git log --oneline --graph</code>	History tab
Open a pull request	the link printed by <code>git push</code> , or <code>gh pr create</code>	Branch → Create pull request

Table 9. Git Bash commands to keep in mind.

Before you leave

After the seminar: the GitHub Skills course "Introduction to GitHub", sections 6 to 10 of the lab notebook saved to the group repository (Part 7), your personal repository with its files from section 9 (Part 8), and your first AI-use journal entry (ask a chatbot to explain `git fetch`, `git pull` and `git merge`, then test each claim in the group repository and record what it got wrong).